Πειραματική Αξιολόγηση της Κατανεμημένης Εκτέλεσης Ερωτημάτων SQL με χρήση Trino σε Ετερογενή Συστήματα Αποθήκευσης Δεδομένων

Ευαγγελία Ευσταθία Παππά 03116759
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εθνικό Μετσόβιο Πολυτεχνείο
Αθήνα, Ελλάδα
pappaevag@gmail.com

Περίληψη—**Ο σκοπός της παρούσας εργασίας εξαμήνου είναι η μελέτη, η εγκατάσταση και η πειραματική αξιολόγηση ενός σύγχρονου περιβάλλοντος διαχείρισης δεδομένων, το οποίο συνδυάζει σχεσιακές βάσεις δεδομένων (PostgreSQL), βάσεις εγγράφων (MongoDB) και μηχανές ευρετηρίασης και αναλυτικών ερωτημάτων (Elasticsearch). Στο πλαίσιο της εργασίας, χρησιμοποιήθηκε το σύνολο δεδομένων TPC-DS για τη δημιουργία και φόρτωση των δεδομένων στα επιμέρους συστήματα. Έπειτα, πραγματοποιήθηκε σύγκριση της απόδοσης τόσο σε native επίπεδο όσο και μέσω του ίδιου του Trino. Στην πράξη, τα πειραματικά αποτελέσματα αναδεικνύουν τα πλεονεκτήματα και τους περιορισμούς της κατανεμημένης επεξεργασίας των ερωτημάτων σε ετερογενή περιβάλλοντα.**

Λέξεις-Κλειδιά: Distributed SQL Queries, Trino, PostgreSQL, MongoDB, Elasticsearch, TPC-DS

I. Εισαγωγή

Ο σκοπός της παρούσας εργασίας εξαμήνου είναι η μελέτη, η εγκατάσταση και η πειραματική αξιολόγηση ενός σύγχρονου περιβάλλοντος διαχείρισης δεδομένων το οποίο ουσιαστικά συνδυάζει βάσεις δεδομένων (PostgreSQL), βάσεις εγγράφων (MongoDB) και μηχανές ευρετηρίασης (Elasticsearch).

Η εργασία καλύπτει το τρίτο σενάριο (Distributed Execution of SQL Queries) της εκφώνησης και επικεντρώνεται στην κατανόηση των τρόπων αποθήκευσης, αναζήτησης και επεξεργασίας των δεδομένων. Κατά την εκπόνηση της εργασίας, αξιοποιήθηκε το σύνολο δεδομένων TPC-DS [1].

II. Βήματα Υλοποίησης

Για την εγκατάσταση και παραμετροποίηση των επιμέρους συστημάτων, αξιοποιήθηκε η τεχνολογία Docker [2] με την οποία δημιουργήθηκαν ορισμένα απομονωμένα containers. Αρχικά πραγματοποιήθηκε η λήψη του επίσημου πακέτου TPC-DS και η αποσυμπίεση του τοπικά (στον host). Στη συνέχεια, δημιουργήθηκε το Docker image που περιλαμβάνει τα απαραίτητα build tools (λ.χ. gcc, make, bison, flex και perl) και κάνει compile τα TPC-DS tools. Για λόγους συμβατότητας (και τεκμηρίωσης) χρησιμοποιήθηκε το βασικό image Ubuntu 20.04 ώστε η διαδικασία του build να ολοκληρωθεί ομαλά.

Στον ριζικό φάκελο του TPC-DS (εκεί που βρίσκονται τα tools/, specification/, κλπ.) δημιουργήθηκε ένα αρχείο Dockerfile με το ακόλουθο περιεχόμενο:

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    build-essential \
    gcc \
    g++ \
    make \
    perl \
    bison \
    flex \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /tpcds
COPY . /tpcds

WORKDIR /tpcds/tools
RUN make -f Makefile.suite OS=LINUX
```

Κατόπιν, το image δημιουργήθηκε με την εντολή:

```
docker build -t tpcds-tools .
```

```
[+] Building 983.6s (11/11) FINISHED                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                              0.0s
 => => transferring dockerfile: 627B                                              0.0s
 => [internal] load metadata for docker.io/library/ubuntu:20.04                   0.8s
 => [internal] load .dockerignore                                                 0.0s
 => => transferring context: 2B                                                   0.0s
 => [1/6] FROM docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8f         3.4s
 => => resolve docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8f         0.0s
 => => sha256:ecd83b6c354452b6a9979c7666bba16927f1e60e2 25.98MB / 25.98MB         3.0s
 => => extracting sha256:ecd83b6c354452b6a9979c7666bba16927f1e60e2afbfe64         0.3s
 => [internal] load build context                                                 0.0s
 => => transferring context: 94.24kB                                              0.0s
 => [2/6] RUN apt-get update && apt-get install -y     build-essential          970.8s
 => [3/6] WORKDIR /tpcds                                                          0.0s
 => [4/6] COPY . /tpcds                                                           0.0s
 => [5/6] WORKDIR /tpcds/tools                                                    0.0s
 => [6/6] RUN make -f Makefile.suite OS=LINUX                                     1.7s
 => exporting to image                                                            6.8s
 => => exporting layers                                                           5.7s
 => => exporting manifest sha256:13a3eef2997e3ffd95c841f94e001dadd8e9ea90         0.0s
 => => exporting config sha256:e0b41cc4388a3b9c3cc19002fd02bcb4fbb34a6f74         0.0s
 => => exporting attestation manifest sha256:e9b78e17c0eab0d87ee025a935e3         0.0s
 => => exporting manifest list sha256:f8189cd4c0200aeaab9a239561f308f1241         0.0s
 => => naming to docker.io/library/tpcds-tools:latest                             0.0s
 => => unpacking to docker.io/library/tpcds-tools:latest                          1.0s
```

Εικόνα 1: Δημιουργία Docker image για το TPC-DS

Με την ολοκλήρωση του build, το image περιέχει τα binaries στη διαδρομή /tpcds/tools/, συμπεριλαμβανομένου του dsdgen (δηλ. του data generator). Για την παραγωγή των δεδομένων δημιουργήθηκε ένα μόνιμο container, με bind-mount σε τοπικό φάκελο, ώστε τα παραγόμενα αρχεία .dat να αποθηκεύονται στον host και να παραμένουν διαθέσιμα ανεξάρτητα από την κατάσταση του container. Ο φάκελος εξόδου δημιουργήθηκε στον host ως ~/tpcds-data/sf2. Στη συνέχεια εκκινήθηκε το container με την παρακάτω εντολή:

```
docker run -it \
  --name tpcds-gen \
  -v ~/tpcds-data:/data \
  tpcds-tools \
  bash
```

Εντός του container, η παραγωγή του dataset για Scale Factor = 2 (δηλ. για ελάχιστα gigabytes) εκτελέστηκε ως εξής:

```
cd /tpcds/tools
mkdir -p /data/sf2
./dsdgen -scale 2 -dir /data/sf2
```

Σε περίπτωση που χρειαστεί να επαναληφθεί η διαδικασία, το container μπορεί να επανεκκινηθεί με την παρακάτω εντολή:

```
docker start -ai tpcds-gen
```

Για να γίνει επιβεβαίωση ότι το dataset (με τα αρχεία) όντως παρήχθη, εκτελέστηκαν οι παρακάτω εντολές στον host:

```
ls -lh ~/tpcds-data/sf2 | head
du -sh ~/tpcds-data/sf2
```

```
ls -lh ~/tpcds-data/sf2 | head


 2.4K Feb  2 13:17 call_center.dat
 1.6M Feb  2 13:17 catalog_page.dat
  41M Feb  2 13:19 catalog_returns.dat
 568M Feb  2 13:19 catalog_sales.dat
  18M Feb  2 13:19 customer.dat
 7.6M Feb  2 13:19 customer_address.dat
  77M Feb  2 13:20 customer_demographics.dat
 9.8M Feb  2 13:20 date_dim.dat
  52B Feb  2 13:36 dbgen_version.dat
du -sh ~/tpcds-data/sf2
```

Εικόνα 2: Παραγόμενα αρχεία δεδομένων από το TPC-DS

Το αμέσως επόμενο στάδιο της εργασίας αφορά την εγκατάσταση και τη ρύθμιση του συστήματος βάσεων δεδομένων, προκειμένου να φορτωθούν τα δεδομένα και να εκτελεστούν τα ερωτήματα σε μορφή SQL. Η εγκατάσταση της PostgreSQL [3] πραγματοποιήθηκε εξίσου με το Docker και συγκεκριμένα μέσω του εργαλείου docker-compose.

Το αρχείο docker-compose.yml περιγράφει το service της βάσης δεδομένων, τις παραμέτρους αρχικοποίησης και τα απαιτούμενα volumes. Το αρχείο έχει την ακόλουθη μορφή:

```
version: "3.9"

services:
  postgres:
    image: postgres:15
    container_name: tpcds-postgres
    environment:
      POSTGRES_USER: tpcds
      POSTGRES_PASSWORD: tpcds
      POSTGRES_DB: tpcds
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ~/tpcds-data/sf2:/data
    shm_size: 1g

volumes:
  pgdata:
```

Η εκκίνηση της βάσης πραγματοποιήθηκε με την εντολή:

```
docker compose up -d
```

Μετά την εκτέλεση της εντολής, το container της εκκινεί σε detached mode. Η ορθή λειτουργία επιβεβαιώθηκε δίνοντας:

```
docker ps | grep tpcds-postgres
```

```
[+] Running 14/14
 ✔ postgres Pulled                                                    16.6s
   ✔ 2a80b8df8f70 Download complete                                    0.7s
   ✔ 49d636a047b4 Download complete                                    1.1s
   ✔ 66cf8f189b0d Download complete                                    0.7s
   ✔ 08724fee5eb4 Download complete                                    4.2s
   ✔ 12cd95217c0c Download complete                                    0.5s
   ✔ 6be5ff533868 Download complete                                    0.9s
   ✔ 00a8b15719db Download complete                                    0.8s
   ✔ 8401b2c82bf9 Download complete                                   13.6s
   ✔ 26f3966302f5 Download complete                                    1.3s
   ✔ 48bb3c3185f6 Download complete                                    0.7s
   ✔ 91bf75daf7f5 Download complete                                    0.7s
   ✔ 0062edb7bb84 Download complete                                    0.7s
   ✔ 34f7c774a5db Download complete                                    2.0s
[+] Running 3/3
 ✔ Network tpcds-postgres_default  Cre...                              0.0s
 ✔ Volume "tpcds-postgres_pgdata"  Cre...                              0.0s
 ✔ Container tpcds-postgres        Started                             0.4s
```

Εικόνα 3: Δημιουργία Docker image για την PostgreSQL

Αφού ολοκληρώθηκε επιτυχώς η εγκατάσταση και εκκίνηση της PostgreSQL, το επόμενο βήμα αφορά τη δημιουργία του σχήματος σύμφωνα με τις προδιαγραφές του TPC-DS. Συγκεκριμένα, δημιουργήθηκαν ορισμένοι πίνακες που αντιστοιχούν στα δεδομένα τα οποία παρήχθησαν από τον TPC-DS data generator (π.χ. οι call_center, customer, customer_address, customer_demographics και date_dim). Οι τύποι δεδομένων προσαρμόστηκαν για την PostgreSQL, χρησιμοποιώντας κατάλληλους τύπους ακεραίων και δεκαδικών. Έπειτα, το σχήμα αποθηκεύτηκε στο αρχείο tpcds_schema.sql το οποίο εκτελέστηκε εντός του container:

```
DROP TABLE IF EXISTS call_center;
DROP TABLE IF EXISTS customer;
DROP TABLE IF EXISTS customer_address;
```

```
DROP TABLE IF EXISTS customer_demographics;
DROP TABLE IF EXISTS date_dim;


CREATE TABLE call_center (
  cc_call_center_sk INTEGER,
  cc_call_center_id CHAR(16),
  cc_rec_start_date DATE,
  cc_rec_end_date DATE,
  cc_closed_date_sk INTEGER,
  cc_open_date_sk INTEGER,
  cc_name VARCHAR(50),
  cc_class VARCHAR(50),
  cc_employees INTEGER,
  cc_sq_ft INTEGER,
  cc_hours CHAR(20),
  cc_manager VARCHAR(40),
  cc_mkt_id INTEGER,
  cc_mkt_class VARCHAR(50),
  cc_mkt_desc VARCHAR(100),
  cc_market_manager VARCHAR(40),
  cc_division INTEGER,
  cc_division_name VARCHAR(50),
  cc_company INTEGER,
  cc_company_name VARCHAR(50),
  cc_street_number VARCHAR(10),
  cc_street_name VARCHAR(60),
  cc_street_type CHAR(15),
  cc_suite_number CHAR(10),
  cc_city VARCHAR(60),
  cc_county VARCHAR(30),
  cc_state CHAR(2),
  cc_zip CHAR(10),
  cc_country VARCHAR(20),
  cc_gmt_offset DECIMAL(5,2),
  cc_tax_percentage DECIMAL(5,2),
  cc_end_date DATE
);

CREATE TABLE customer (
  c_customer_sk INTEGER,
  c_customer_id CHAR(16),
  c_current_cdemo_sk INTEGER,
  c_current_hdemo_sk INTEGER,
  c_current_addr_sk INTEGER,
  c_first_shipto_date_sk INTEGER,
  c_first_sales_date_sk INTEGER,
  c_salutation CHAR(10),
  c_first_name CHAR(20),
  c_last_name CHAR(30),
  c_preferred_cust_flag CHAR(1),
  c_birth_day INTEGER,
  c_birth_month INTEGER,
  c_birth_year INTEGER,
  c_birth_country VARCHAR(20),
  c_login CHAR(13),
  c_email_address CHAR(50),
  c_last_review_date_sk INTEGER,
  c_customer_since DATE
);

CREATE TABLE customer_address (
  ca_address_sk INTEGER,
  ca_address_id CHAR(16),
  ca_street_number CHAR(10),
  ca_street_name VARCHAR(60),
  ca_street_type CHAR(15),
  ca_suite_number CHAR(10),
  ca_city VARCHAR(60),
  ca_county VARCHAR(30),
  ca_state CHAR(2),
  ca_zip CHAR(10),
  ca_country VARCHAR(20),
  ca_gmt_offset DECIMAL(5,2),
  ca_location_type CHAR(20),
  ca_address_since DATE
);

CREATE TABLE customer_demographics (
  cd_demo_sk INTEGER,
  cd_gender CHAR(1),
  cd_marital_status CHAR(1),
  cd_education_status CHAR(20),
  cd_purchase_estimate INTEGER,
  cd_credit_rating CHAR(10),
  cd_dep_count INTEGER,
  cd_dep_employed_count INTEGER,
  cd_dep_college_count INTEGER,
  cd_demo_since DATE
);

CREATE TABLE date_dim (
  d_date_sk INTEGER,
  d_date_id CHAR(16),
  d_date DATE,
  d_month_seq INTEGER,
  d_week_seq INTEGER,
  d_quarter_seq INTEGER,
  d_year INTEGER,
  d_dow INTEGER,
  d_moy INTEGER,
  d_dom INTEGER,
  d_qoy INTEGER,
  d_fy_year INTEGER,
  d_fy_quarter_seq INTEGER,
  d_fy_week_seq INTEGER,
  d_day_name CHAR(9),
  d_quarter_name CHAR(6),
  d_holiday CHAR(1),
  d_weekend CHAR(1),
  d_following_holiday CHAR(1),
  d_first_dom INTEGER,
  d_last_dom INTEGER,
  d_same_day_ly INTEGER,
  d_same_day_lq INTEGER,
  d_current_day CHAR(1),
  d_current_week CHAR(1),
  d_current_month CHAR(1),
  d_current_quarter CHAR(1),
  d_current_year CHAR(1),
  d_date_since DATE
);
```

Για την φόρτωση, δόθηκε η εντολή:

```
docker exec -it tpcds-postgres \
  psql -U tpcds -d tpcds -f
/data/tpcds_schema.sql
```

```
psql (15.15 (Debian 15.15-1.pgdg13+1))
Type "help" for help.

tpcds=# \dt
                List of relations
 Schema |         Name          | Type  | Owner
--------+-----------------------+-------+-------
 public | call_center           | table | tpcds
 public | customer              | table | tpcds
 public | customer_address      | table | tpcds
 public | customer_demographics | table | tpcds
 public | date_dim              | table | tpcds
(5 rows)

tpcds=#
```

Εικόνα 4: Επιβεβαίωση σχήματος στην PostgreSQL

Η εισαγωγή των δεδομένων πραγματοποιήθηκε με τη χρήση της εντολής COPY της PostgreSQL. Τα αρχεία ήταν σε μορφή pipe-delimited (|) και δεν περιείχαν επικεφαλίδες:

```
\set ON_ERROR_STOP on

COPY call_center
FROM '/data/call_center.dat'
WITH (FORMAT csv, DELIMITER '|', NULL '');

COPY customer
FROM '/data/customer.dat'
WITH (FORMAT csv, DELIMITER '|', NULL '');

COPY customer_address
FROM '/data/customer_address.dat'
WITH (FORMAT csv, DELIMITER '|', NULL '');

COPY customer_demographics
FROM '/data/customer_demographics.dat'
WITH (FORMAT csv, DELIMITER '|', NULL '');

COPY date_dim
FROM '/data/date_dim.dat'
WITH (FORMAT csv, DELIMITER '|', NULL '');
```

```
/data/load_5_tables.sql

COPY 8
COPY 144000
COPY 72000
COPY 1920800
COPY 73049
```

Εικόνα 5: Εισαγωγή δεδομένων στην PostgreSQL

Για την επιβεβαίωση των εισαχθέντων δεδομένων:

```
SELECT
  'call_center' AS table, COUNT(*) FROM
call_center
UNION ALL
SELECT 'customer', COUNT(*) FROM customer
UNION ALL
SELECT 'customer_address', COUNT(*) FROM
customer_address
UNION ALL
SELECT 'customer_demographics', COUNT(*)
FROM customer_demographics
UNION ALL
SELECT 'date_dim', COUNT(*) FROM date_dim;
```

```
/data/check_5_tables.sql

          table          |  count
-------------------------+---------
 call_center             |       8
 customer                |  144000
 customer_address        |   72000
 customer_demographics   | 1920800
 date_dim                |   73049
(5 rows)
```

Εικόνα 6: Επιβεβαίωση δεδομένων στην PostgreSQL

Για την εκτίμηση της απόδοσης, χρησιμοποιήθηκε η εντολή EXPLAIN ANALYZE η οποία εκτελεί το εκάστοτε query και επιστρέφει τον συνολικό χρόνο που απαιτήθηκε:

```
EXPLAIN ANALYZE
SELECT
  c_birth_year,
  c_birth_country,
  COUNT(*) AS customer_count
FROM customer
GROUP BY c_birth_year, c_birth_country
ORDER BY c_birth_year, customer_count DESC;

EXPLAIN ANALYZE
SELECT
  cd_education_status,
  cd_marital_status,
  AVG(cd_purchase_estimate) AS avg_estimate,
  COUNT(*) AS cnt
FROM customer c
JOIN customer_demographics cd
  ON c.c_current_cdemo_sk = cd.cd_demo_sk
WHERE cd.cd_purchase_estimate > 5000
GROUP BY cd_education_status,
cd_marital_status
ORDER BY avg_estimate DESC;

EXPLAIN ANALYZE
SELECT
  ca_state,
  ca_country,
  COUNT(*) AS customers
FROM customer c
JOIN customer_address ca
  ON c.c_current_addr_sk = ca.ca_address_sk
GROUP BY ca_state, ca_country
ORDER BY customers DESC;
```

Τα αποτελέσματα είχαν ως εξής:

```
----------------------------------------------------------------------
----------------------------------------------------------
 Sort  (cost=7644.59..7680.59 rows=14400 width=21) (actual time=44.66
3..45.110 rows=14837 loops=1)
   Sort Key: c_birth_year, (count(*)) DESC
   Sort Method: quicksort  Memory: 1334kB
   ->  HashAggregate  (cost=6506.00..6650.00 rows=14400 width=21) (ac
tual time=39.803..40.958 rows=14837 loops=1)
         Group Key: c_birth_year, c_birth_country
         Batches: 1  Memory Usage: 2065kB
         ->  Seq Scan on customer  (cost=0.00..5426.00 rows=144000 wi
dth=13) (actual time=0.010..7.659 rows=144000 loops=1)
 Planning Time: 0.245 ms
 Execution Time: 45.717 ms
(9 rows)
```

Εικόνα 7: Ανάλυση πρώτου ερωτήματος σε PostgreSQL

```
----------------------------------------------------------------------
----------------------------------------------------------------------
-----------------------------------
 Sort  (cost=41980.16..41980.24 rows=35 width=63) (actual time=81.772
..82.932 rows=35 loops=1)
   Sort Key: (avg(cd.cd_purchase_estimate)) DESC
   Sort Method: quicksort  Memory: 28kB
   ->  Finalize GroupAggregate  (cost=41969.95..41979.26 rows=35 widt
h=63) (actual time=81.712..82.917 rows=35 loops=1)
         Group Key: cd.cd_education_status, cd.cd_marital_status
         ->  Gather Merge  (cost=41969.95..41978.12 rows=70 width=63)
 (actual time=81.705..82.892 rows=105 loops=1)
               Workers Planned: 2
               Workers Launched: 2
               ->  Sort  (cost=40969.93..40970.02 rows=35 width=63) (
actual time=80.588..80.590 rows=35 loops=3)
                     Sort Key: cd.cd_education_status, cd.cd_marital_
status
                     Sort Method: quicksort  Memory: 28kB
                     Worker 0:  Sort Method: quicksort  Memory: 28kB
                     Worker 1:  Sort Method: quicksort  Memory: 28kB
                     ->  Partial HashAggregate  (cost=40968.68..40969
.03 rows=35 width=63) (actual time=80.557..80.562 rows=35 loops=3)
                           Group Key: cd.cd_education_status, cd.cd_m
arital_status
                           Batches: 1  Memory Usage: 24kB
                           Worker 0:  Batches: 1  Memory Usage: 24kB
                           Worker 1:  Batches: 1  Memory Usage: 24kB
                           ->  Parallel Hash Join  (cost=5336.00..406
78.64 rows=29004 width=27) (actual time=12.414..76.955 rows=23113 loo
ps=3)
                                 Hash Cond: (cd.cd_demo_sk = c.c_curr
ent_cdemo_sk)
                                 ->  Parallel Seq Scan on customer_de
mographics cd  (cost=0.00..33718.17 rows=400967 width=31) (actual tim
e=0.042..40.274 rows=320133 loops=3)
                                       Filter: (cd_purchase_estimate
> 5000)
                                       Rows Removed by Filter: 320133
                                 ->  Parallel Hash  (cost=4586.00..45
86.00 rows=60000 width=4) (actual time=12.178..12.178 rows=48000 loop
s=3)
                                       Buckets: 262144  Batches: 1  M
emory Usage: 7552kB
                                       ->  Parallel Seq Scan on custo
mer c  (cost=0.00..4586.00 rows=60000 width=4) (actual time=0.003..5.
995 rows=48000 loops=3)
 Planning Time: 0.143 ms
 Execution Time: 82.976 ms
(28 rows)
```

Εικόνα 8: Ανάλυση δεύτερου ερωτήματος σε PostgreSQL

Στην πράξη:

- Το πρώτο ερώτημα (ομαδοποίηση πελατών ανά έτος γέννησης και χώρα) εκτελέστηκε σε 45.7 ms.
- Το δεύτερο (join customer-customer_demographics με φίλτρο cd_purchase_estimate > 5000 και aggregation ανά κατηγορία εκπαίδευσης/ οικογενειακής κατάστασης) εκτελέστηκε σε 83.0 ms.
- Το τρίτο (join customer–customer_address και aggregation ανά πολιτεία/χώρα) εκτελέστηκε σε 28.5 ms, με parallel hash join και in-memory sort.

```
----------------------------------------------------------------------
----------------------------------------------------------------------
---------------------------------
 Sort  (cost=9217.48..9217.61 rows=51 width=25) (actual time=26.733..
28.462 rows=102 loops=1)
   Sort Key: (count(*)) DESC
   Sort Method: quicksort  Memory: 31kB
   ->  Finalize GroupAggregate  (cost=9202.86..9216.04 rows=51 width=
25) (actual time=26.625..28.443 rows=102 loops=1)
         Group Key: ca.ca_state, ca.ca_country
         ->  Gather Merge  (cost=9202.86..9214.76 rows=102 width=25)
(actual time=26.619..28.392 rows=299 loops=1)
               Workers Planned: 2
               Workers Launched: 2
               ->  Sort  (cost=8202.84..8202.97 rows=51 width=25) (ac
tual time=25.625..25.630 rows=100 loops=3)
                     Sort Key: ca.ca_state, ca.ca_country
                     Sort Method: quicksort  Memory: 31kB
                     Worker 0:  Sort Method: quicksort  Memory: 31kB
                     Worker 1:  Sort Method: quicksort  Memory: 31kB
                     ->  Partial HashAggregate  (cost=8200.88..8201.3
9 rows=51 width=25) (actual time=25.565..25.574 rows=100 loops=3)
                           Group Key: ca.ca_state, ca.ca_country
                           Batches: 1  Memory Usage: 40kB
                           Worker 0:  Batches: 1  Memory Usage: 40kB
                           Worker 1:  Batches: 1  Memory Usage: 40kB
                           ->  Parallel Hash Join  (cost=2586.94..775
0.88 rows=60000 width=17) (actual time=5.891..19.472 rows=48000 loops
=3)
                                 Hash Cond: (c.c_current_addr_sk = ca
.ca_address_sk)
                                 ->  Parallel Seq Scan on customer c
 (cost=0.00..4586.00 rows=60000 width=4) (actual time=0.003..2.769 ro
ws=48000 loops=3)
                                 ->  Parallel Hash  (cost=2057.53..20
57.53 rows=42353 width=21) (actual time=5.756..5.756 rows=24000 loops
=3)
                                       Buckets: 131072  Batches: 1  M
emory Usage: 4992kB
                                       ->  Parallel Seq Scan on custo
mer_address ca  (cost=0.00..2057.53 rows=42353 width=21) (actual time
=0.004..2.421 rows=24000 loops=3)
 Planning Time: 0.091 ms
 Execution Time: 28.499 ms
(26 rows)
```

Εικόνα 9: Ανάλυση τρίτου ερωτήματος σε PostgreSQL

Ακολούθως, δημιουργήθηκε container για την MongoDB [4]:

```
services:
  mongo:
    image: mongo:7
    container_name: tpcds-mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

volumes:
  mongo_data:
```

```
[+] Running 8/8
 ✔ mongo Pulled                                                    33.6s
   ✔ 58e38be1fd4a Download complete                                 0.5s
   ✔ ce84a8e6c25a Download complete                                 0.4s
   ✔ 5e004b4024d5 Download complete                                28.5s
   ✔ 02b7af02ed07 Download complete                                 0.6s
   ✔ 9e0465564662 Download complete                                 0.7s
   ✔ fd33fc5eb885 Download complete                                 0.7s
   ✔ c29253f4fae0 Download complete                                 0.8s
[+] Running 3/3
```

Εικόνα 10: Δημιουργία Docker image για την MongoDB

Η μεταφορά των δεδομένων έγινε με:

```
docker exec -it tpcds-postgres psql -U tpcds
-d tpcds -c "\copy (
  SELECT
```

```
    c.c_customer_sk,
    c.c_customer_id,
    c.c_birth_day, c.c_birth_month,
c.c_birth_year, c.c_birth_country,
    c.c_login, c.c_email_address,
    cd.cd_gender, cd.cd_marital_status,
cd.cd_education_status,
cd.cd_purchase_estimate,
cd.cd_credit_rating,
    ca.ca_city, ca.ca_county, ca.ca_state,
ca.ca_zip, ca.ca_country
  FROM customer c
  LEFT JOIN customer_demographics cd ON
c.c_current_cdemo_sk = cd.cd_demo_sk
  LEFT JOIN customer_address ca ON
c.c_current_addr_sk = ca.ca_address_sk
) TO '/tmp/customer_profile.csv' WITH CSV
HEADER"
```

```
pcds -c "\copy (
  SELECT
    c.c_customer_sk,
    c.c_customer_id,
    c.c_birth_day, c.c_birth_month, c.c_birth_year, c.c_birth_country,
    c.c_login, c.c_email_address,
    cd.cd_gender, cd.cd_marital_status, cd.cd_education_status, cd.cd_purchase_e
stimate, cd.cd_credit_rating,
    ca.ca_city, ca.ca_county, ca.ca_state, ca.ca_zip, ca.ca_country
  FROM customer c
  LEFT JOIN customer_demographics cd ON c.c_current_cdemo_sk = cd.cd_demo_sk
  LEFT JOIN customer_address ca ON c.c_current_addr_sk = ca.ca_address_sk
) TO '/tmp/customer_profile.csv' WITH CSV HEADER"

COPY 144000
```

Εικόνα 11: Μεταφορά δεδομένων σε MongoDB

```
  --db tpcds --collection customer_profiles \
  --type csv --headerline \
  --file /tmp/customer_profile.csv

2026-02-02T15:04:24.084+0000    connected to: mongodb://localhost/
2026-02-02T15:04:26.689+0000    144000 document(s) imported successfully. 0 docu
ment(s) failed to import.
```

Εικόνα 12: Επιβεβαίωση μεταφοράς σε MongoDB

Έπειτα, εκτελέστηκαν τα τρία ερωτήματα:

```
var t=Date.now();
db.customer_profiles.aggregate([
  {$group:{
    _id:{year:"$c_birth_year",
country:"$c_birth_country"},
    cnt:{$sum:1}
  }},
  {$sort:{"_id.year":1, cnt:-1}}
]).toArray();
print("ms:", Date.now()-t);

var t=Date.now();
db.customer_profiles.aggregate([

{$match:{cd_purchase_estimate:{$gt:5000}}},
  {$group:{
    _id:{
      education:"$cd_education_status",
      marital:"$cd_marital_status"
    },

avg_purchase:{$avg:"$cd_purchase_estimate"}
  }},
  {$sort:{avg_purchase:-1}}
]).toArray();
print("ms:", Date.now()-t);

var t=Date.now();
db.customer_profiles.aggregate([
  {$group:{
    _id:{state:"$ca_state",
country:"$ca_country"},
    customers:{$sum:1}
  }},
  {$sort:{customers:-1}}
]).toArray();
print("ms:", Date.now()-t);

db.customer_profiles.createIndex({c_birth_ye
ar:1, c_birth_country:1})
db.customer_profiles.createIndex({cd_educati
on_status:1, cd_marital_status:1,
cd_purchase_estimate:1})
db.customer_profiles.createIndex({ca_state:1
, ca_country:1})
```

```
tpcds> var t=Date.now();
... db.customer_profiles.aggregate([
...   {$group:{
...     _id:{year:"$c_birth_year", country:"$c_birth_country"},
...     cnt:{$sum:1}
...   }},
...   {$sort:{"_id.year":1, cnt:-1}}
... ]).toArray();
... print("ms:", Date.now()-t);
...
ms: 176
```

Εικόνα 13: Ανάλυση πρώτου ερωτήματος σε MongoDB

```
tpcds> var t=Date.now();
... db.customer_profiles.aggregate([
...   {$match:{cd_purchase_estimate:{$gt:5000}}},
...   {$group:{
...     _id:{
...       education:"$cd_education_status",
...       marital:"$cd_marital_status"
...     },
...     avg_purchase:{$avg:"$cd_purchase_estimate"}
...   }},
...   {$sort:{avg_purchase:-1}}
... ]).toArray();
... print("ms:", Date.now()-t);
...
ms: 89
```

Εικόνα 14: Ανάλυση δεύτερου ερωτήματος σε MongoDB

```
tpcds> var t=Date.now();
... db.customer_profiles.aggregate([
...   {$group:{
...     _id:{state:"$ca_state", country:"$ca_country"},
...     customers:{$sum:1}
...   }},
...   {$sort:{customers:-1}}
... ]).toArray();
... print("ms:", Date.now()-t);
...
ms: 119
```

Εικόνα 15: Ανάλυση τρίτου ερωτήματος σε MongoDB

Τα τρία ερωτήματα σε MongoDB χρειάστηκαν περίπου 176, 89 και 119ms αντίστοιχα.

Πλέον, για ElasticSearch [5], χρειάζεται νέο container (υπάρχει έτοιμο image από το αποθετήριο του Docker):

```
docker run -d \
  --name tpcds-elastic \
  -p 9200:9200 \
  -e "discovery.type=single-node" \
  -e "xpack.security.enabled=false" \

docker.elastic.co/elasticsearch/elasticsearc
h:8.12.0
```

```
  --name tpcds-elastic \
  -p 9200:9200 \
  -e "discovery.type=single-node" \
  -e "xpack.security.enabled=false" \
  docker.elastic.co/elasticsearch/elasticsearch:8.12.0

Unable to find image 'docker.elastic.co/elasticsearch/elasticsearch:8.12.0' locally
8.12.0: Pulling from elasticsearch/elasticsearch
53f0b8a39cc3: Download complete
62d74e4d0713: Download complete
6d7a592ac56c: Download complete
0ce2a11a08ce: Download complete
2ab7a6e51ec2: Download complete
89732bc75041: Download complete
9325c6c5f8c7: Download complete
b9f9879d0742: Download complete
64d9583ad409: Download complete
2a5b9801e955: Download complete
Digest: sha256:ec72548cf833e58578d8ff40df44346a49480b2c88a4e73a91e1b85ec7ef0d44
Status: Downloaded newer image for docker.elastic.co/elasticsearch/elasticsearch:8.12.0
c259924d88dbd0f2c711e6c41030d691fe2304970e82e2dde24bce85d1ffd06f
```

Εικόνα 16: Λήψη Docker image για την ElasticSearch

Τα δεδομένα ετοιμάστηκαν σε μορφή NDJSON, σύμφωνα με τις απαιτήσεις του ElasticSearch Bulk API. Ουσιαστικά, κάθε εγγραφή αντιμετωπίστηκε ως ένα ανεξάρτητο JSON document. Η εισαγωγή των δεδομένων πραγματοποιήθηκε με χρήση του _bulk endpoint. Μετά την ολοκλήρωση της φόρτωσης, τα δεδομένα ήταν άμεσα διαθέσιμα για την εκτέλεση των aggregation queries μέσω του _search endpoint.

Πριν τη φόρτωση των δεδομένων, δημιουργήθηκε το index customer_profiles με explicit mappings για αριθμητικά και κατηγορικά πεδία, , ώστε να υποστηρίζονται τα aggregations.

```
curl -X PUT localhost:9200/customer_profiles
\
  -H "Content-Type: application/json" \
  -d '{
    "mappings": {
      "properties": {
        "c_birth_year": { "type": "integer"
},
        "c_birth_country": { "type":
"keyword" },
        "cd_purchase_estimate": { "type":
"integer" },
        "cd_education_status": { "type":
"keyword" },
        "cd_marital_status": { "type":
"keyword" },
        "ca_state": { "type": "keyword" },
        "ca_country": { "type": "keyword" }
      }
    }
  }'
```

Η εισαγωγή πραγματοποιήθηκε με μία εντολή curl:

```
curl -X POST localhost:9200/_bulk \
  -H "Content-Type: application/x-ndjson" \
  --data-binary
@/Users/test/bulk_fixed_compact.ndjson
```


```
{"errors":true,"took":4105,"items":[{"index":{"_index":"customer_profiles","_i
w8m5ZbqHDD9","_version":1,"result":"created","_shards":{"total":2,"successful"
},"_seq_no":0,"_primary_term":1,"status":201}},{"index":{"_index":"customer_pr
:"q3ABH5wBxw8m5ZbqHDD-","_version":1,"result":"created","_shards":{"total":2,"
,"failed":0},"_seq_no":1,"_primary_term":1,"status":201}},{"index":{"_index":"
iles","_id":"rHABH5wBxw8m5ZbqHDD-","_version":1,"result":"created","_shards":{
ccessful":1,"failed":0},"_seq_no":2,"_primary_term":1,"status":201}},{"index":
stomer_profiles","_id":"rXABH5wBxw8m5ZbqHDD-","_version":1,"result":"created",
otal":2,"successful":1,"failed":0},"_seq_no":3,"_primary_term":1,"status":201}
_index":"customer_profiles","_id":"rnABH5wBxw8m5ZbqHDD-","_version":1,"result"
shards":{"total":2,"successful":1,"failed":0},"_seq_no":4,"_primary_term":1,"s
{"index":{"_index":"customer_profiles","_id":"r3ABH5wBxw8m5ZbqHDD-","_version"
created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":5,"_primary
tus":201}},{"index":{"_index":"customer_profiles","_id":"sHABH5wBxw8m5ZbqHDD-"
,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":
erm":1,"status":201}},{"index":{"_index":"customer_profiles","_id":"sXABH5wBxw
_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":0}
"_primary_term":1,"status":201}},{"index":{"_index":"customer_profiles","_id":
5ZbqHDD-","_version":1,"result":"created","_shards":{"total":2,"successful":1,
_seq_no":8,"_primary_term":1,"status":201}},{"index":{"_index":"customer_profi
3ABH5wBxw8m5ZbqHDD-","_version":1,"result":"created","_shards":{"total":2,"suc
ailed":0},"_seq_no":9,"_primary_term":1,"status":201}},{"index":{"_index":"cus
s","_id":"tHABH5wBxw8m5ZbqHDD-","_version":1,"result":"created","_shards":{"to
ssful":1,"failed":0},"_seq_no":10,"_primary_term":1,"status":201}},{"index":{"
omer_profiles","_id":"tXABH5wBxw8m5ZbqHDD-","_version":1,"result":"created","_
al":2,"successful":1,"failed":0},"_seq_no":11,"_primary_term":1,"status":201}}
index":"customer_profiles","_id":"tnABH5wBxw8m5ZbqHDD-","_version":1,"result":
hards":{"total":2,"successful":1,"failed":0},"_seq_no":12,"_primary_term":1,"s
{"index":{"_index":"customer_profiles","_id":"t3ABH5wBxw8m5ZbqHDD-","_version"
created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":13,"_primar
atus":201}},{"index":{"_index":"customer_profiles","_id":"uHABH5wBxw8m5ZbqHDD-
1,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no"
_term":1,"status":201}},{"index":{"_index":"customer_profiles","_id":"uXABH5wB
,"_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":
15,"_primary_term":1,"status":201}},{"index":{"_index":"customer_profiles","_i
w8m5ZbqHDD-","_version":1,"result":"created","_shards":{"total":2,"successful"
},"_seq_no":16,"_primary_term":1,"status":201}},{"index":{"_index":"customer_p
```


Εικόνα 17: Μαζική φόρτωση δεδομένων στην ElasticSearch

Για την επιβεβαίωση του πλήθους των εγγράφων:

```
curl -X GET
localhost:9200/customer_profiles/_count
```


```
{"count":142067,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0}}%
```


Εικόνα 18: Επιβεβαίωση δεδομένων στην ElasticSearch

Τα τρία ερωτήματα εκτελέστηκαν ως εξής:

```
curl -s -X POST
localhost:9200/customer_profiles/_search \
  -H "Content-Type: application/json" \
  -d '{
    "size": 0,
    "aggs": {
      "by_year": {
        "terms": { "field": "c_birth_year",
"size": 20 },
        "aggs": {
          "by_country": {
            "terms": { "field":
"c_birth_country", "size": 10 }
          }
        }
      }
    }
  }' | jq '.took'
```

```
curl -s -X POST
localhost:9200/customer_profiles/_search \
  -H "Content-Type: application/json" \
  -d '{
    "size": 0,
    "query": {
      "range": { "cd_purchase_estimate": {
"gt": 5000 } }
    },
    "aggs": {
      "by_edu": {
        "terms": { "field":
"cd_education_status", "size": 50 },
        "aggs": {
          "by_marital": {
            "terms": { "field":
"cd_marital_status", "size": 10 },
            "aggs": {
              "avg_purchase": {
                "avg": { "field":
"cd_purchase_estimate" }
              }
            }
          }
        }
      }
    }
  }' | jq '.took'

curl -s -X POST
localhost:9200/customer_profiles/_search \
  -H "Content-Type: application/json" \
  -d '{
    "size": 0,
    "aggs": {
      "by_country": {
        "terms": { "field": "ca_country",
"size": 50 },
        "aggs": {
          "by_state": {
            "terms": { "field": "ca_state",
"size": 100 }
          }
        }
      }
    }
  }' | jq '.took'
```

Από τις εκτελέσεις στην ElasticSearch (με μέτρηση μέσω του took) προκύπτει ότι τα queries ολοκληρώθηκαν σε 38, 58 και 49ms αντίστοιχα. Το δεύτερο ερώτημα ήταν το πιο απαιτητικό γιατί συνδυάζει range filter (cd_purchase_estimate > 5000) με πολυεπίπεδο grouping και υπολογισμό μέσου όρου (avg) και αυτό αυξάνει το συνολικό «κόστος» της επεξεργασίας.

```
  -H "Content-Type: application/json" \
  -d '{
    "size": 0,
    "aggs": {
      "by_year": {
        "terms": { "field": "c_birth_year", "size": 20 },
        "aggs": {
          "by_country": {
            "terms": { "field": "c_birth_country", "size": 10 }
          }
        }
      }
    }
  }' | jq '.took'
38
```

Εικόνα 19: Ανάλυση πρώτου ερωτήματος σε ElasticSearch

```
  -H "Content-Type: application/json" \
  -d '{
    "size": 0,
    "query": {
      "range": { "cd_purchase_estimate": { "gt": 5000 } }
    },
    "aggs": {
      "by_edu": {
        "terms": { "field": "cd_education_status", "size": 50 },
        "aggs": {
          "by_marital": {
            "terms": { "field": "cd_marital_status", "size": 10 },
            "aggs": {
              "avg_purchase": {
                "avg": { "field": "cd_purchase_estimate" }
              }
            }
          }
        }
      }
    }
  }' | jq '.took'
58
```

Εικόνα 20: Ανάλυση δεύτερου ερωτήματος σε ElasticSearch

```
  -H "Content-Type: application/json" \
  -d '{
    "size": 0,
    "aggs": {
      "by_country": {
        "terms": { "field": "ca_country", "size": 50 },
        "aggs": {
          "by_state": {
            "terms": { "field": "ca_state", "size": 100 }
          }
        }
      }
    }
  }' | jq '.took'
49
```

Εικόνα 21: Ανάλυση τρίτου ερωτήματος σε ElasticSearch

Τέλος, όσον αφορά το Trino [6], δημιουργήθηκε container:

```
version: "3.8"

services:
  trino:
    image: trinodb/trino:455
    container_name: trino
    ports:
      - "8080:8080"
    volumes:
      - ./trino/catalog:/etc/trino/catalog
```

Επίσης, για τη μεταφορά των δεδομένων από το υπάρχον περιβάλλον με PostgreSQL, δημιουργήθηκε το εξής αρχείο:

```
mkdir -p trino/catalog
nano trino/catalog/postgres.properties
```

Με τις παρακάτω παραμέτρους:

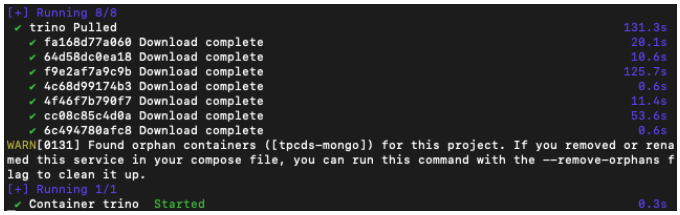
```
connector.name=postgresql
connection-
url=jdbc:postgresql://host.docker.internal:5
432/tpcds
connection-user=tpcds
connection-password=tpcds
```

Αντίστοιχα, για σύνδεση με MongoDB, δημιουργείται το:

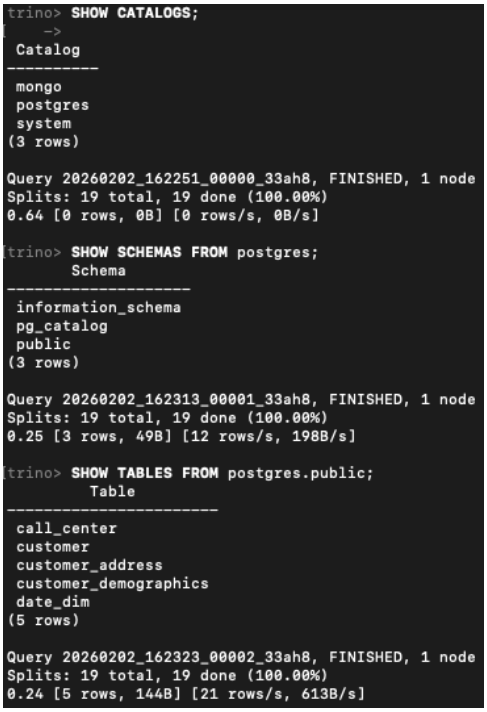
```
nano trino/catalog/mongo.properties
```

Με τις παρακάτω παραμέτρους:

```
connector.name=mongodb
mongodb.connection-url=mongodb://mongo:27017
```



Εικόνα 22: Λήψη Docker image για το Trino

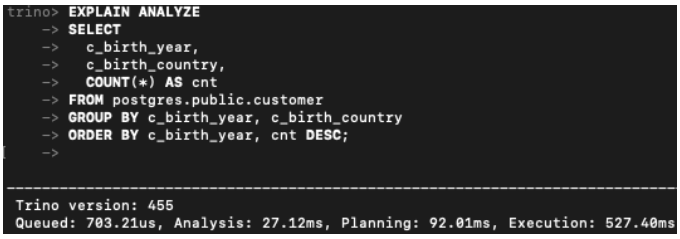


Εικόνα 23: Επιβεβαίωση μεταφοράς των δεδομένων στο Trino

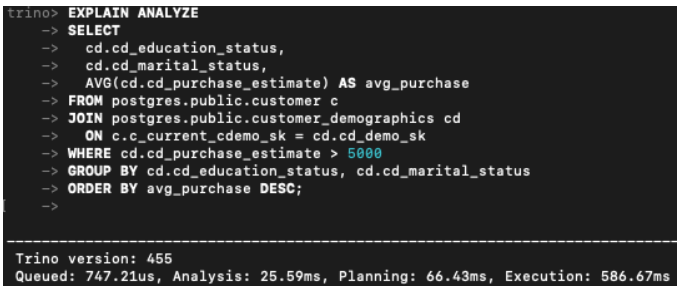
Τα ερωτήματα σε Trino εκτελέστηκαν ως εξής (PostgreSQL):

```
EXPLAIN ANALYZE
SELECT
  c_birth_year,
  c_birth_country,
  COUNT(*) AS cnt
FROM postgres.public.customer
GROUP BY c_birth_year, c_birth_country
ORDER BY c_birth_year, cnt DESC;

EXPLAIN ANALYZE
SELECT
  cd.cd_education_status,
  cd.cd_marital_status,
  AVG(cd.cd_purchase_estimate) AS
avg_purchase
FROM postgres.public.customer c
JOIN postgres.public.customer_demographics
cd
  ON c.c_current_cdemo_sk = cd.cd_demo_sk
WHERE cd.cd_purchase_estimate > 5000
GROUP BY cd.cd_education_status,
cd.cd_marital_status
ORDER BY avg_purchase DESC;

EXPLAIN ANALYZE
SELECT
  ca.ca_country,
  ca.ca_state,
  COUNT(*) AS customers
FROM postgres.public.customer c
JOIN postgres.public.customer_address ca
  ON c.c_current_addr_sk = ca.ca_address_sk
GROUP BY ca.ca_country, ca.ca_state
ORDER BY customers DESC;
```

Παρότι οι χρόνοι εκτέλεσης ήταν αρκετά υψηλότεροι (527.4, 586.67 και 447.39ms), το Trino είναι ένα ισχυρό εργαλείο για την ενοποιημένη ανάλυση των δεδομένων (που προέρχονται από διαφορετικές πηγές) κάτω από ένα ενιαίο SQL interface.



Εικόνα 24: Ανάλυση πρώτου ερωτήματος σε Trino (Postgre)



Εικόνα 25: Ανάλυση δεύτερου ερωτήματος σε Trino (Postgre)

```
trino> EXPLAIN ANALYZE
    -> SELECT
    ->   ca.ca_country,
    ->   ca.ca_state,
    ->   COUNT(*) AS customers
    -> FROM postgres.public.customer c
    -> JOIN postgres.public.customer_address ca
    ->   ON c.c_current_addr_sk = ca.ca_address_sk
    -> GROUP BY ca.ca_country, ca.ca_state
    -> ORDER BY customers DESC;
    ->

Trino version: 455
Queued: 209.83us, Analysis: 21.15ms, Planning: 48.74ms, Execution: 447.39ms
```

Εικόνα 26: Ανάλυση τρίτου ερωτήματος σε Trino (Postgre)

Έπειτα, πάλι στο Trino, αλλά για τα δεδομένα από MongoDB:

```
EXPLAIN ANALYZE
SELECT
  c_birth_year,
  c_birth_country,
  COUNT(*) AS cnt
FROM mongo.tpcds.customer_profiles
GROUP BY c_birth_year, c_birth_country
ORDER BY c_birth_year, cnt DESC;

EXPLAIN ANALYZE
SELECT
  cd_education_status,
  cd_marital_status,
  AVG(cd_purchase_estimate) AS avg_purchase
FROM mongo.tpcds.customer_profiles
WHERE cd_purchase_estimate > 5000
GROUP BY cd_education_status,
cd_marital_status
ORDER BY avg_purchase DESC;

EXPLAIN ANALYZE
SELECT
  ca_country,
  ca_state,
  COUNT(*) AS customers
FROM mongo.tpcds.customer_profiles
GROUP BY ca_country, ca_state
ORDER BY customers DESC;
```

```
trino> EXPLAIN ANALYZE
    -> SELECT
    ->   c_birth_year,
    ->   c_birth_country,
    ->   COUNT(*) AS cnt
    -> FROM mongo.tpcds.customer_profiles
    -> GROUP BY c_birth_year, c_birth_country
    -> ORDER BY c_birth_year, cnt DESC;
    ->

Trino version: 455
Queued: 770.08us, Analysis: 47.71ms, Planning: 32.46ms, Execution: 624.43ms
```

Εικόνα 27: Ανάλυση πρώτου ερωτήματος σε Trino (Mongo)

```
trino> EXPLAIN ANALYZE
    -> SELECT
    ->   cd_education_status,
    ->   cd_marital_status,
    ->   AVG(cd_purchase_estimate) AS avg_purchase
    -> FROM mongo.tpcds.customer_profiles
    -> WHERE cd_purchase_estimate > 5000
    -> GROUP BY cd_education_status, cd_marital_status
    -> ORDER BY avg_purchase DESC;
    ->

Trino version: 455
Queued: 822.42us, Analysis: 10.99ms, Planning: 30.04ms, Execution: 342.80ms
```

Εικόνα 28: Ανάλυση δεύτερου ερωτήματος σε Trino (Mongo)

```
trino> EXPLAIN ANALYZE
    -> SELECT
    ->   ca_country,
    ->   ca_state,
    ->   COUNT(*) AS customers
    -> FROM mongo.tpcds.customer_profiles
    -> GROUP BY ca_country, ca_state
    -> ORDER BY customers DESC;
    ->

Trino version: 455
Queued: 433.46us, Analysis: 3.98ms, Planning: 19.54ms, Execution: 328.12ms
```

Εικόνα 29: Ανάλυση τρίτου ερωτήματος σε Trino (Mongo)

III. ΣΥΜΠΕΡΑΣΜΑΤΑ

Οι χρόνοι εκτέλεσης για τα 3 αντιπροσωπευτικά ερωτήματα:

Ερώτημα	PostgreSQL	MongoDB	Elastic Search	Trino (PostgreSQL)	Trino (MongoDB)
Κατανομή πελατών ανά έτος γέννησης & χώρα	~45 ms	~176 ms	**38 ms**	~527 ms	~624 ms
Μέσο purchase estimate ανά education & marital status	~83 ms	~89 ms	**58 ms**	~586 ms	~342 ms
Πλήθος πελατών ανά χώρα & πολιτεία	~28 ms	~119 ms	**49 ms**	~447 ms	~328 ms

Συμπερασματικά:

- Η PostgreSQL παρουσιάζει πολύ καλή απόδοση σε αναλυτικά ερωτήματα πάνω σε κανονικοποιημένα σχήματα, ειδικά για aggregations και joins.
- Η MongoDB πετυχαίνει αντίστοιχους χρόνους σε ομαδοποιήσεις χωρίς joins, αλλά υστερεί όταν αυξάνεται η πολυπλοκότητα των aggregations.

- Η Elasticsearch εμφανίζει τους καλύτερους χρόνους σε aggregation-heavy ερωτήματα, χάρη στην έμφαση που δίνει σε analytics και search workloads.

- Το Trino έχει σημαντικά υψηλότερους χρόνους λόγω του overhead της μεταφοράς και της επεξεργασίας των δεδομένων από εξωτ. πηγές, ωστόσο προσφέρει το βασικό πλεονέκτημα μιας ενοποιημένης διεπαφής για την ανάλυση πολλαπλών συστημάτων.

Στο Trino, παρατηρείται αισθητά καλύτερη απόδοση όταν η πηγή δεδομένων είναι η MongoDB (αντί για PostgreSQL). Αυτό μπορεί να αποδοθεί στον μεγαλύτερο βαθμό aggregation pushdown και στο αποκανονικοποιημένο σχήμα δεδομένων της MongoDB. Επομένως, είναι σημαντικό να επιλέγεται η κατάλληλη storage engine σε περιβάλλοντα με distributed queries.

IV. ΒΙΒΛΙΟΓΡΑΦΙΑ

[1] TPC-DS Benchmark Version 4.0.0, Transaction Processing Performance Council (TPC), 2024. Διαθέσιμο: https://www.tpc.org/tpcds/

[2] Docker, Docker Documentation, 2026. Διαθέσιμο: https://docs.docker.com/

[3] The PostgreSQL Global Development Group, PostgreSQL Documentation, 2026. Διαθέσιμο: https://www.postgresql.org/docs/

[4] MongoDB, MongoDB Docs, 2026. Διαθέσιμο: https://www.mongodb.com/docs/

[5] Elastic, Elasticsearch Guide, 2025. Διαθέσιμο: https://www.elastic.co/guide/en/elasticsearch/reference/8.19/index.html

[6] Trino Software Foundation, Trino Documentation, 2025. Διαθέσιμο: https://trino.io/docs/current/